

# **Modelagem dimensional de vendas e ETL incremental com AdventureWorks**

ANDRÉ ALVES DA SILVA

BRENO DOS SANTOS GUIMARÃES

LUCAS DE LIMA NASCIMENTO

SERGIO PAULO DE ANDRADE

Centro Universitário Salesiano - UniSales

Curso: Sistemas de Informação

Disciplina: Projeto de Análise e Fluxo de Dados - OLAP e ETL

Professor: James Alves

Vitória, 2026

## **RESUMO**

Este trabalho apresenta a construção de um modelo dimensional de vendas baseado no banco transacional AdventureWorks2016, com destino em PostgreSQL e processo de extração, transformação e carga incremental desenvolvido em Python. O objetivo é demonstrar a relação entre granularidade, definição de indicadores e consistência da integração de dados. O modelo segue o padrão estrela, com uma tabela fato no nível de item de pedido e seis dimensões independentes. Foram definidos dez indicadores relacionados a valores comerciais, volume, clientes, variação mensal e expedição. A estratégia incremental utiliza Change Tracking do SQL Server para identificar chaves inseridas, alteradas ou excluídas, incluindo dependências entre tabelas que formam dimensões desnormalizadas. A leitura ocorre em snapshot consistente e a confirmação dos dados e do checkpoint é atômica no destino. A validação em bases isoladas aprovou 28 testes, incluindo operações reais de Change Tracking. No AdventureWorks restaurado, a reconciliação comparou 121.317 itens na origem e no DW, sem divergências. As consultas retornaram receita líquida de 109.846.381,399888 UM e 31.465 pedidos expedidos; os retornos dos dez indicadores foram incorporados ao artigo. O estudo demonstra como decisões explícitas de granularidade, população analítica e recuperação evitam duplicação de medidas e perda silenciosa de alterações.

Palavras-chave: Data Warehouse; modelagem dimensional; modelo estrela; ETL incremental; AdventureWorks.

## **ABSTRACT**

This work presents a dimensional sales model based on the AdventureWorks2016 transactional database, using PostgreSQL as the target and Python for incremental extraction, transformation and loading. Its objective is to demonstrate the relationship between fact grain, performance metrics and integration consistency. The design follows a star schema with one fact table at sales order line grain and six independent dimensions. Ten metrics cover commercial amounts, sales volume, customers, monthly change and shipping. SQL Server Change Tracking identifies inserted, updated and deleted keys, including dependencies across source tables used to build denormalized dimensions. Source reads use a consistent snapshot, while target data and synchronization checkpoints are committed atomically. Validation in isolated databases passed 28 tests, including actual Change Tracking operations. Reconciliation against the restored AdventureWorks backup matched 121,317 source and warehouse items with no differences. Queries returned net sales of 109,846,381.399888 source monetary units and 31,465 shipped orders; outputs of all ten metrics are included in this paper. The study illustrates how explicit decisions about grain, analytical scope and failure recovery prevent duplicated measures and silent loss of changes.

Keywords: Data Warehouse; dimensional modeling; star schema; incremental ETL; AdventureWorks.

## **1 INTRODUÇÃO**

Sistemas transacionais registram operações como pedidos, produtos e clientes, organizando os dados para preservar integridade durante inclusões e alterações. Perguntas analíticas, entretanto, combinam grandes conjuntos de registros e exigem definições consistentes para períodos, populações e medidas. No contexto de vendas, uma consulta pode produzir valores numericamente plausíveis e ainda assim estar incorreta se somar o total de um pedido para cada um de seus itens.

O problema investigado neste trabalho é como transformar o domínio de vendas do AdventureWorks em uma estrutura multidimensional que suporte dez indicadores e permaneça sincronizada sem recarregar indiscriminadamente todas as tabelas. A modelagem e a integração são tratadas de forma conjunta: o nível de detalhe determina quais medidas podem ser armazenadas, enquanto as dependências entre tabelas determinam quais registros precisam ser reavaliados após uma mudança na origem.

O objetivo geral é implementar um data mart de vendas em PostgreSQL seguindo o padrão Star Schema, alimentado por uma ETL incremental em Python. Os objetivos específicos são analisar as entidades do OLTP, declarar a granularidade, selecionar dimensões, formalizar os indicadores, tratar registros novos e alterados, prever exclusões e demonstrar a recuperação de falhas. A documentação e os testes complementam o código para permitir sua reprodução e defesa técnica.

O AdventureWorks é um conjunto de bases de exemplo disponibilizado pela Microsoft. Para este projeto foi selecionado o backup OLTP completo de 2016, restaurado no SQL Server 2022, evitando utilizar a base AdventureWorksDW já modelada para análise ou a versão LT simplificada (Microsoft, s.d.a). O recorte abrange vendas de produtos; compras, estoque e fabricação não integram o modelo analítico proposto.

## 2 FUNDAMENTAÇÃO TEÓRICA

### 2.1 DATA WAREHOUSE E MODELAGEM DIMENSIONAL

Um Data Warehouse integra dados para consulta analítica com regras de interpretação estáveis. Um data mart delimita essa finalidade a um processo ou domínio, como vendas. A disponibilidade de dados em um banco separado, por si só, não garante qualidade analítica: é necessário definir a origem de cada medida, o evento representado por uma linha e o comportamento de agregação.

Na modelagem dimensional, fatos representam medições e dimensões descrevem os contextos de análise. O grão deve ser declarado antes da escolha de medidas e dimensões, pois registros de níveis diferentes não devem coexistir indistintamente na mesma fato (Kimball Group, s.d.). Neste trabalho, essa orientação significa distinguir item, pedido e cliente: uma linha mede a venda de um produto dentro de um pedido, enquanto pedidos e clientes são contados de forma distinta.

O modelo estrela liga uma fato central diretamente a dimensões descritivas. Hierarquias podem ser armazenadas na própria dimensão, reduzindo a necessidade de o usuário percorrer estruturas relacionais normalizadas (Kimball, 2003). No domínio de produtos, categoria e subcategoria são incorporadas a dim\_produto. Não há uma dimensão categoria referenciada por uma dimensão subcategoria: essa opção manteria parte da hierarquia em floco de neve, contrariando o padrão escolhido para o trabalho.

### 2.2 ADITIVIDADE E ALTERAÇÕES DE DIMENSÕES

Uma medida aditiva pode ser somada nos eixos pertinentes ao seu grão. Quantidade e valor de itens são exemplos no recorte de vendas. Taxas, preços e médias exigem outra abordagem: a soma de percentuais não expressa a taxa global e a média simples de tickets de grupos com tamanhos diferentes pode distorcer o resultado. Por isso, os indicadores derivados são calculados a partir de numeradores e denominadores agregados.

Chaves substitutas permitem identificar membros dimensionais independentemente da codificação operacional. Nas dimensões de produto, cliente, território e vendedor, a chave de negócio permanece única para apoiar o lookup e o upsert, enquanto a chave substituta é referenciada pela fato. A dimensão data usa uma chave determinística no formato YYYYMMDD e a dimensão canal possui dois membros estáticos.

Descrições de dimensões podem mudar com o tempo. A estratégia Slowly Changing Dimension tipo 1 sobrescreve o atributo, enquanto a tipo 2 cria versões para preservar sua vigência histórica (Kimball, 2008). Adotou-se SCD1 porque os indicadores propostos usam descrições atuais. A consequência é explícita: renomear uma categoria altera a classificação exibida também para vendas antigas. Isso não deve ser interpretado como histórico completo da classificação no momento da venda.

## 2.3 EXTRAÇÃO TRANSFORMAÇÃO E CARGA INCREMENTAL

A ETL extrai dados da origem, aplica regras de transformação e os carrega no destino. A incrementalidade delimita cada ciclo aos registros novos, alterados, excluídos e às dependências necessárias para manter o modelo consistente. Um mecanismo de detecção é insuficiente sem uma política de checkpoint, pois uma falha entre a escrita dos dados e a atualização do marcador pode causar perda ou repetição de trabalho.

O Change Tracking do SQL Server permite consultar chaves modificadas desde uma versão conhecida. Para uma leitura consistente, a documentação orienta validar a versão mínima, obter a versão atual e consultar as mudanças dentro de uma transação com isolamento snapshot (Microsoft, s.d.b). O mecanismo fornece o estado necessário à sincronização, sem representar um histórico completo de todas as versões intermediárias.

No PostgreSQL, a instrução INSERT com ON CONFLICT permite resolver a presença de uma chave já existente por atualização (PostgreSQL Global Development Group, s.d.). Neste projeto, o upsert é condicionado à diferença material entre os valores, e a confirmação ocorre na mesma transação que grava o checkpoint. A integração admite repetição de leitura após falha, mas preserva a unicidade do estado final por item.

## 3 DESENVOLVIMENTO E MÉTODO

### 3.1 ANÁLISE DO ADVENTUREWORKS OLTP

Sales.SalesOrderHeader contém os atributos do pedido, incluindo cliente, vendedor, território, status e datas. Sales.SalesOrderDetail contém produto, quantidade, preço, desconto e LineTotal. Um cabeçalho pode possuir vários detalhes. A ligação entre essas tabelas fornece a base transacional da fato, com identificação composta por SalesOrderID e SalesOrderDetailID.

Production.Product relaciona-se a ProductSubcategory, que se relaciona a ProductCategory. Sales.Customer identifica clientes vinculados a Person.Person ou Sales.Store. O vendedor utiliza Sales.SalesPerson e Person.Person, enquanto Sales.SalesTerritory fornece nome, código de país e grupo comercial. Campos opcionais são tratados deliberadamente, evitando que joins internos indevidos removam vendas sem vendedor ou território associado.

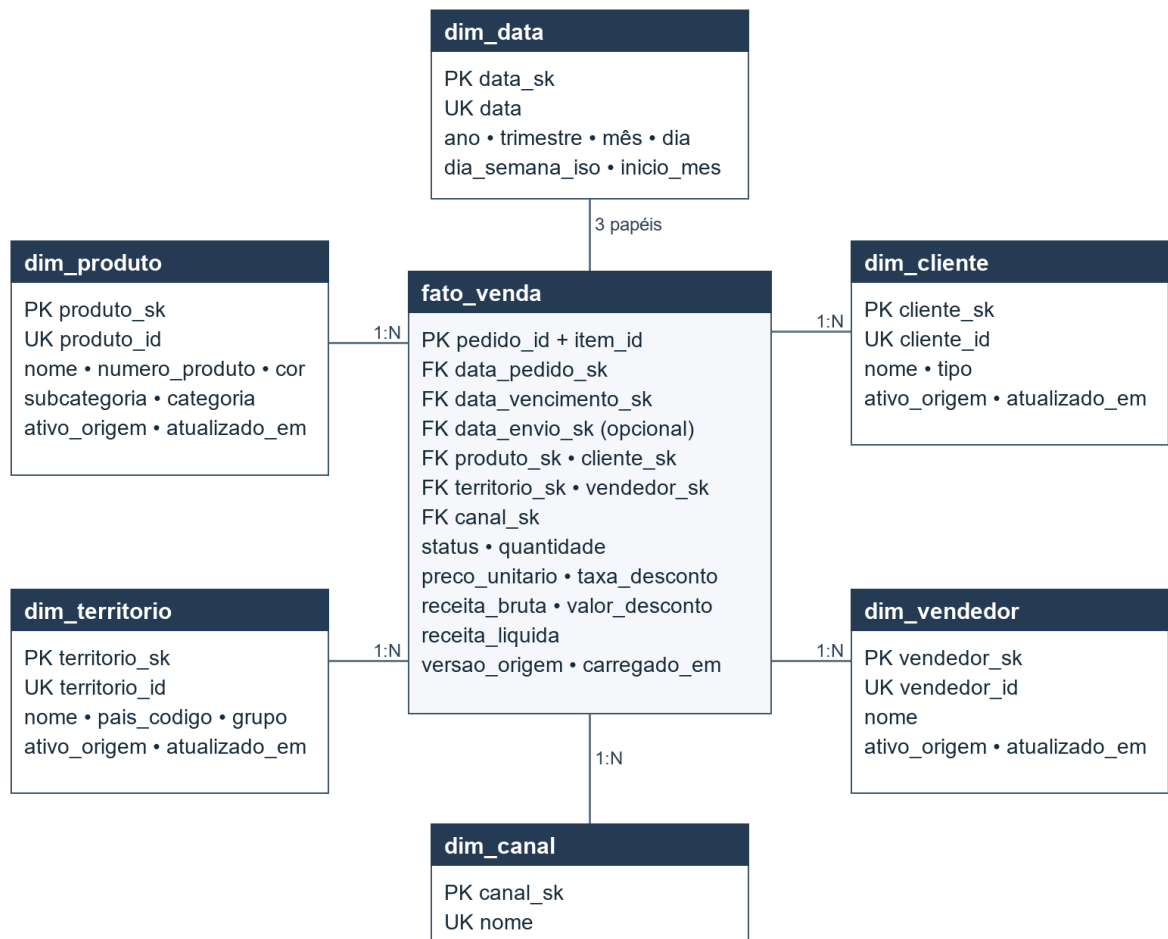
O catálogo real do backup deve confirmar os relacionamentos e tipos utilizados. Para isso, foram implementadas consultas a sys.tables, sys.columns, sys.foreign\_keys e propriedades descritivas, além de perfis de status, datas e medidas. A figura relacional de 2008 disponibilizada no enunciado serve como apoio à leitura, mas não substitui a inspeção da versão 2016. O script de análise também compara subtotais e soma de itens, distinguindo arredondamento de erro de agregação.

### 3.2 MODELO ESTRELA E GRANULARIDADE

A fato\_venda possui uma linha por item de pedido no estado mais recente sincronizado. Sua chave primária composta é pedido\_id e item\_id. O pedido\_id também funciona como dimensão degenerada, permitindo localizar o documento comercial sem criar uma tabela descritiva de apenas um identificador. A fato armazena status, quantidade, preço, taxa de desconto e receita líquida, além das chaves das dimensões.

São utilizadas seis dimensões: dim\_data, dim\_produto, dim\_cliente, dim\_territorio, dim\_vendedor e dim\_canal. A dimensão data participa pelos papéis pedido, vencimento e envio, com três FKs independentes. As outras cinco dimensões possuem uma FK cada, totalizando oito relacionamentos dimensionais na fato. Não há FKs entre dimensões, preservando o padrão estrela. O diagrama apresenta essa organização.

Figura 1 - Modelo estrela de vendas



Grão: um item de pedido • Sem relações entre dimensões • SCD1 nas descrições

Fonte: elaboração própria (2026).

Produto e cliente são obrigatórios. Vendedor e território ausentes na origem utilizam membros técnicos com SK zero. A data de envio permanece nula se não informada; inventar uma data alteraria o indicador de prazo. Se uma chave

obrigatória não for encontrada no lookup, o lote falha e é revertido. O dicionário completo de colunas, tipos, origens e regras encontra-se no Apêndice A.

### 3.3 REGRAS COMERCIAIS E INDICADORES

Os dez indicadores adotam a população de pedidos expedidos, identificada por Status igual a 5, e usam a data do pedido como eixo temporal padrão. A fato mantém os demais status para refletir alterações posteriores. Esse recorte representa uma convenção comercial didática e não uma política contábil de reconhecimento de receita.

Receita bruta é quantidade multiplicada pelo preço unitário; receita líquida comercial é LineTotal; o desconto monetário corresponde à diferença entre ambas. Frete, tributos e TotalDue pertencem ao grão pedido e não são repetidos como medidas aditivas nos itens. Não se calcula lucro com StandardCost atual, pois isso não demonstraria o custo histórico efetivo da venda.

Quadro 1 - Indicadores implementados

| Nº | Indicador                  | Regra principal                                                            |
|----|----------------------------|----------------------------------------------------------------------------|
| 1  | Receita bruta              | Soma de quantidade × preço                                                 |
| 2  | Descontos comerciais       | Soma de bruta - líquida                                                    |
| 3  | Receita líquida comercial  | Soma de LineTotal                                                          |
| 4  | Unidades vendidas          | Soma de quantidade                                                         |
| 5  | Pedidos expedidos          | Contagem distinta de pedido_id                                             |
| 6  | Ticket médio               | Receita líquida ÷ pedidos distintos                                        |
| 7  | Taxa ponderada de desconto | $100 \times \text{descontos} \div \text{receita bruta}$                    |
| 8  | Clientes com compra        | Contagem distinta de cliente_sk                                            |
| 9  | Crescimento mensal         | Variação frente ao mês calendário anterior                                 |
| 10 | Expedição até vencimento   | $100 \times \text{pedidos pontuais} \div \text{pedidos com data de envio}$ |

Fonte: elaboração própria (2026).

Os valores monetários são conservados em unidades monetárias da fonte, sem nova conversão cambial. A agregação pressupõe a convenção homogênea de UnitPrice e LineTotal no conjunto didático, que deve ser confirmada na análise do backup. Se uma fonte mantiver valores em moedas locais heterogêneas, será necessária uma modelagem adicional de moeda e conversão antes de utilizar os totais monetários.

As razões usam NULLIF no denominador para representar situações indefinidas sem erro de divisão por zero. A série mensal inclui meses intermediários sem venda antes de aplicar LAG; assim, março não é comparado diretamente com janeiro quando fevereiro está vazio. Contagens de clientes e pedidos distintos não podem ser somadas indiscriminadamente entre categorias ou períodos.

O décimo indicador deduplica pedidos e compara ShipDate com DueDate. Ele é denominado expedição até o vencimento, pois os campos não comprovam a entrega ao cliente. Pedidos expedidos sem data de envio são informados separadamente e não entram no denominador. Os scripts completos estão no Apêndice B e o documento KPIS.md do projeto detalha objetivos, relevância, linhagem, interpretação e argumentos de defesa para cada indicador.

### 3.4 IMPLEMENTAÇÃO DO DW E DA ETL

O PostgreSQL implementa PKs, FKs, unicidade das chaves de negócio e verificações de domínio. As medidas monetárias usam numeric e a transformação Python usa Decimal. Colunas geradas calculam receita bruta e desconto monetário, evitando que diferentes escritores mantenham fórmulas incompatíveis. Índices apoiam os principais filtros dimensionais, sem afirmar ganho de desempenho que ainda não foi medido no backup real.

No início de cada carga, uma trava de sessão no PostgreSQL impede concorrência entre execuções do pipeline. A tabela etl.execucao registra a tentativa e etl.controle fornece a última versão confirmada. A origem inicia uma transação snapshot, verifica a validade do checkpoint em todas as tabelas monitoradas e captura a versão superior. Mudanças confirmadas depois desse snapshot ficam para a próxima execução. A tabela temporária de pedidos afetados e seu índice são criados antes do início do snapshot, enquanto a leitura das mudanças ocorre dentro da transação consistente; essa organização corrige o erro SQL Server 3964.

As chaves retornadas pelo CT formam conjuntos de dependências. Uma alteração no cabeçalho exige reavaliar os itens do pedido; uma alteração de categoria exige reavaliar os produtos associados. Mudanças em Person alcançam tanto clientes quanto vendedores. Dessa forma, a desnormalização do modelo é acompanhada por regras explícitas de propagação, sem depender apenas do ModifiedDate da entidade principal.

O lote é materializado em memória e a transação de leitura da origem é encerrada antes da escrita no destino. Dentro de uma única transação PostgreSQL, a ETL atualiza dimensões por SCD1, completa o calendário, resolve chaves e faz COPY dos itens para staging temporário. Valida as medidas e identifica itens ausentes nos pedidos afetados para tratar exclusões. O upsert grava somente linhas novas ou materialmente diferentes; linhas inalteradas reavaliadas por dependência não são regravadas.

Por fim, a ETL compara os itens do staging com o destino, grava o checkpoint e marca sucesso na mesma transação. Se ocorrer falha antes do COMMIT, dados e marcador são revertidos. Se a confirmação ocorrer antes de uma interrupção do processo, o checkpoint já corresponde aos dados. Não há transação distribuída porque a origem é somente lida; a garantia necessária concentra-se na confirmação conjunta dos dados e do marcador no destino.

### 3.5 RETENÇÃO RECUPERAÇÃO E REPRODUTIBILIDADE

O CT é preparado com retenção de sete dias. Quando o checkpoint é inferior à versão mínima válida, o programa interrompe a carga e exige ressincronização explícita por run --resync. A leitura completa de recuperação preserva as chaves substitutas, atualiza dimensões, elimina fatos ausentes e confirma um novo marcador. Essa exceção não substitui os ciclos incrementais regulares.

A identidade da origem combina um identificador criado na preparação da base e versões de habilitação do CT. O recurso ajuda a detectar troca de origem, mas não garante identificar toda restauração de backup. Após restauração ou alteração administrativa do rastreamento, o procedimento determina ressincronização completa. CT sincroniza o estado atual e não preserva todas as mudanças intermediárias.

Docker Compose descreve SQL Server, PostgreSQL e a aplicação Python, com volumes persistentes e verificações de prontidão. A preparação restaura o backup sem sobrescrever bases existentes. O repositório inclui scripts de análise, criação do DW, indicadores, demonstração, testes e documentação. O DBeaver é utilizado para consultar catálogos, visualizar relações e exportar resultados, sem desenvolvimento de dashboard.

Projeto da ETL no GitHub: <https://github.com/lucaslnasc/adventureworks-dw>. A revisão do código executado é identificada pelos hashes SHA-256 de cada arquivo no campo files\_sha256 de evidence/validacao.json. O registro permite comparar os arquivos avaliados com o código disponibilizado no repositório, independentemente do commit que vier a reunir a documentação. O README apresenta o roteiro de instalação e a documentação técnica detalha modelagem, dicionário e estratégia incremental.

## 4 RESULTADOS E DISCUSSÃO

### 4.1 EVIDÊNCIAS DA CARGA E RECONCILIAÇÃO

A revisão automatizada executada em 12 de setembro de 2026 produziu novas exportações: evidence/reconciliation.json às 15h41min53s e evidence/kpis.json às 15h41min54s, horário de Brasília (UTC-3). Esses arquivos acompanham o projeto e substituem, como referência documental desta edição, as transcrições anteriores de 12h57. Ambas as exportações registram a versão CT 0; esse número representa o checkpoint de mudanças, e não a quantidade de linhas carregadas.

O comando verify comparou 121.317 itens na origem e 121.317 no destino, sem divergências de chaves ou atributos de fato. A diferença de zeros finais na representação da receita decorre da escala decimal. A Tabela 1 apresenta os campos exportados.

Tabela 1 - Reconciliação do AdventureWorks

| <b>Campo</b>                        | <b>Retorno</b>     |
|-------------------------------------|--------------------|
| Itens na origem                     | 121317             |
| Itens no DW                         | 121317             |
| Itens divergentes                   | 0                  |
| Receita da origem - todos os status | 109846381.399888   |
| Receita do DW - todos os status     | 109846381.39988800 |
| Versão de sincronização             | 0                  |
| Resultado                           | PASS               |

Fonte: evidence/reconciliation.json; 12/09/2026, 15:41:53 (UTC-3), versão CT 0. Valores em UM; separador decimal do JSON preservado.

A reconciliação considera todos os status, enquanto as views dos indicadores filtram status 5. O perfil da origem mostrou 31.465 pedidos, todos expedidos, explicando a coincidência numérica entre os dois recortes nesta execução. A comparação adicional das descrições de produto, cliente, território e vendedor encontrou zero diferenças. As três primeiras consultas de qualidade retornaram zero linhas.

#### 4.2 RETORNOS DOS DEZ INDICADORES

As dez views foram executadas pelo comando report. A Tabela 2 resume os resultados; o Apêndice C associa cada consulta ao retorno, incluindo a série completa de 38 meses. Os valores provêm do AdventureWorks restaurado, e não da massa sintética dos testes.

Tabela 2 - Indicadores do AdventureWorks

| Nº | Indicador                  | Resultado            |
|----|----------------------------|----------------------|
| 1  | Receita bruta              | 110.373.889,31 UM    |
| 2  | Descontos comerciais       | 527.507,91 UM        |
| 3  | Receita líquida comercial  | 109.846.381,40 UM    |
| 4  | Unidades vendidas          | 274.914              |
| 5  | Pedidos expedidos          | 31.465               |
| 6  | Ticket médio               | 3.491,07 UM          |
| 7  | Taxa ponderada de desconto | 0,4779%              |
| 8  | Clientes com compra        | 19.119               |
| 9  | Crescimento mensal         | 38 meses; Apêndice C |
| 10 | Expedição até vencimento   | 100,0000%            |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0. Síntese arredondada; valores completos no Apêndice C.

A receita bruta de 110.373.889,31340000 UM menos 527.507,91351200 UM de descontos resulta em receita líquida de 109.846.381,39988800 UM. A divisão pelos 31.465 pedidos distintos produz ticket médio de 3.491,0656729664071190 UM. A taxa de desconto global, aproximadamente 0,4779%, é calculada sobre valores agregados.

A série mensal vai de maio de 2011 a junho de 2014. O primeiro mês não tem base anterior e retorna NULL. Março de 2014 apresenta receita de 7.217.531,091974 UM e crescimento de aproximadamente 439,5377%; junho de 2014 apresenta 49.005,840000 UM e variação de -99,0868%. A cobertura e a composição dos dados devem ser examinadas antes de atribuir essas variações a causas de negócio.

O indicador de expedição retorna 31.465 pedidos com data, zero sem data e 31.465 expedidos até o vencimento, totalizando 100%. A comparação entre ShipDate e DueDate não comprova recebimento pelo cliente.

#### 4.3 INTEGRIDADE REFERENCIAL E TESTES DA ETL

O catálogo PostgreSQL confirmou oito chaves estrangeiras com convalidated igual a true. São cinco referências não temporais e três papéis da dimensão data. A Tabela 3 apresenta a consulta registrada em evidence/validacao.json.

```
SELECT conname, convalidated FROM pg_constraint
WHERE conrelid = 'dw.fato_venda'::regclass
AND contype = 'f' ORDER BY conname;
```

Tabela 3 - Chaves estrangeiras da fato

| conname                            | convalidated |
|------------------------------------|--------------|
| fato_venda_canal_sk_fkey           | true         |
| fato_venda_cliente_sk_fkey         | true         |
| fato_venda_data_envio_sk_fkey      | true         |
| fato_venda_data_pedido_sk_fkey     | true         |
| fato_venda_data_vencimento_sk_fkey | true         |
| fato_venda_produto_sk_fkey         | true         |
| fato_venda_territorio_sk_fkey      | true         |
| fato_venda_vendedor_sk_fkey        | true         |

Fonte: catálogo PostgreSQL registrado em evidence/validacao.json, 12/09/2026.

A suíte aprovou 28 testes em 9,90 segundos, sem falhas e sem testes pulados: 13 casos de transformação, 12 de PostgreSQL e três de SQL Server/Change Tracking. Foram verificados indicadores, meses sem venda, denominadores vazios, idempotência, inserção, atualização, exclusão, alteração de cabeçalho, SCD1, rollback, recuperação, auditoria, concorrência e mudanças confirmadas durante snapshot. O teste de regressão do erro 3964 confirmou o ciclo vazio, a deduplicação de pedidos afetados e a exclusão do último item.

A execução utilizou Python 3.13.5 no Windows, pytest 9.1.1, psycpg 3.3.5, pymssql 2.4.1, PostgreSQL 17.11 e SQL Server 2022 16.0.4275.2. As bases sintéticas de teste foram criadas isoladamente e removidas ao final. A imagem declarada no Dockerfile usa Python 3.12; o build Docker e uma nova restauração não foram reexecutados nesta revisão. Os resultados não devem ser apresentados como teste de uma instalação Docker nova.

O histórico da base principal registra carga inicial de 121.317 fatos, falhas incrementais anteriores e depois um ciclo incremental bem-sucedido com zero fatos extraídos, escritos e excluídos. A extração adicional desta revisão, a partir do checkpoint 0, retornou zero fatos, pedidos afetados e membros dimensionais. Essa última checagem foi de leitura, sem nova escrita de lote no DW principal.

Na massa sintética, a receita bruta foi 310 UM, os descontos 20 UM e a receita líquida 290 UM. O mês intermediário sem vendas foi preenchido com zero e a divisão por base anterior zero permaneceu indefinida. Um dos dois pedidos estava no prazo, resultando em 50%. Esses exemplos controlados são distintos dos valores reais apresentados nas tabelas e no Apêndice C.

#### 4.4 LIMITAÇÕES E INTERPRETAÇÃO

SCD1 apresenta as descrições atuais para vendas antigas; CT sincroniza o estado atual, sem fornecer todos os eventos intermediários. A seleção incremental pode reler itens inalterados dos pedidos afetados, mas o upsert só regrava fatos materialmente diferentes. A extração inicial em memória é adequada ao porte didático, podendo exigir staging persistente em maior escala. Não foram realizados benchmarks comparativos de desempenho.

O perfil da origem identificou 13.976 pedidos com CurrencyRateID. O projeto conserva UnitPrice e LineTotal em UM, sem reaplicar câmbio apenas pela existência dessa referência. A reconciliação demonstra fidelidade aos campos da fonte, mas não demonstra, isoladamente, sua convenção monetária; a comparabilidade permanece uma premissa explícita. Expedição até o vencimento também não equivale a entrega ao cliente. Novas fontes ou perguntas de negócio podem exigir moeda, medidas convertidas e novos marcos logísticos.

#### 5 CONSIDERAÇÕES FINAIS

O trabalho apresentou um modelo estrela de vendas, implementado em PostgreSQL e alimentado por ETL incremental em Python. A definição do grão item preserva produtos, quantidades e descontos, exigindo contagem distinta de pedidos e exclusão de medidas repetidas de cabeçalho. A desnormalização foi acompanhada de regras de dependência para atualizar descrições de categorias, lojas e pessoas.

A integração combina versão de mudanças, snapshot consistente, upserts por chaves únicas e checkpoint transacional. Os 28 testes aprovados incluem Change Tracking real e recuperação de falha. No AdventureWorks restaurado, 121.317 itens foram reconciliados sem divergências, e os dez indicadores tiveram seus retornos registrados. O repositório é identificado na seção 3.5 e as evidências anexas identificam os arquivos de código avaliados.

Como evolução, o modelo pode receber SCD2 para descrições históricas, tratamento explícito de múltiplas moedas e processamento em lotes para maior escala. Essas mudanças devem decorrer de novas necessidades e preservar o grão, a consistência e as garantias de recuperação.

## REFERÊNCIAS

CENTRO UNIVERSITÁRIO SALESIANO. Guia de elaboração e normalização de trabalhos acadêmicos e de pesquisa. Vitória: UniSales, 2024. Disponível em: <https://unisales.br/wp-content/uploads/2024/07/NOVO-GUIA-DE-ELABORACAO-E-NORMALIZACAO-DE-TRABALHOS-ACADEMICOS-E-DE-PESQUISA-29.05.pdf>. Acesso em: 12 set. 2026.

KIMBALL, Ralph. Fact tables and dimension tables. Kimball Group, 2003. Disponível em: <https://www.kimballgroup.com/2003/01/fact-tables-and-dimension-tables/>. Acesso em: 12 set. 2026.

KIMBALL, Ralph. Slowly changing dimensions. Kimball Group, 2008. Disponível em: <https://www.kimballgroup.com/2008/08/slowly-changing-dimensions/>. Acesso em: 12 set. 2026.

KIMBALL GROUP. Grain. [S. l.], [s. d.]. Disponível em: <https://www.kimballgroup.com/data-warehouse-business-intelligence-resources/kimball-techniques/dimensional-modeling-techniques/grain/>. Acesso em: 12 set. 2026.

MICROSOFT. AdventureWorks sample databases. Microsoft Learn, [s. d.a]. Disponível em: <https://learn.microsoft.com/en-us/sql/samples/adventureworks-install-configure>. Acesso em: 12 set. 2026.

MICROSOFT. Work with Change Tracking. Microsoft Learn, [s. d.b]. Disponível em: <https://learn.microsoft.com/en-us/sql/relational-databases/track-changes/work-with-change-tracking-sql-server>. Acesso em: 12 set. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. INSERT. PostgreSQL 17 documentation, [s. d.]. Disponível em: <https://www.postgresql.org/docs/17/sql-insert.html>. Acesso em: 12 set. 2026.

## APÊNDICE A - DICIONÁRIO DO DATA WAREHOUSE

PK identifica a linha; NK é a chave da origem; SK é a chave substituta; FK é a referência dimensional. As dimensões descritivas usam SCD1. Os tipos e a nulabilidade correspondem ao DDL e ao catálogo verificado.

Quadro 2 - dw.dim\_canal

| Campo    | Tipo / nulo    | Origem e significado                                                     |
|----------|----------------|--------------------------------------------------------------------------|
| canal_sk | smallint / Não | PK estática 0 revendedor e 1 online, derivada de Header.OnlineOrderFlag. |
| nome     | text / Não     | Rótulo estático Revendedor ou Online.                                    |

Fonte: elaboração própria (2026), com base no DDL e no dicionário do projeto.

Quadro 3 - dw.dim\_cliente

| Campo         | Tipo / nulo                    | Origem e significado                                                                                        |
|---------------|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| cliente_sk    | bigint / Não                   | PK substituta; FK da fato, preservada no SCD1.                                                              |
| cliente_id    | integer / Não                  | NK única Sales.Customer.CustomerID; distingue pessoa ou loja compradora.                                    |
| nome          | text / Não                     | Sales.Store.Name priorizado; senão nome completo de Person.Person ligado por Customer.PersonID.             |
| tipo          | text / Não                     | Loja se StoreID preenchido, Pessoa se PersonID preenchido; caso contrário Não informado.                    |
| ativo_origem  | boolean / Não                  | Metadado SCD1: membro existe atualmente na origem. Não é KPI de atividade comercial.                        |
| atualizado_em | timestamp with time zone / Não | Instante UTC-equivalente armazenado como timestamptz da última gravação no DW; não é ModifiedDate da fonte. |

Fonte: elaboração própria (2026), com base no DDL e no dicionário do projeto.

Quadro 4 - dw.dim\_data

| <b>Campo</b>   | <b>Tipo / nulo</b> | <b>Origem e significado</b>                                                                      |
|----------------|--------------------|--------------------------------------------------------------------------------------------------|
| data_sk        | integer / Não      | PK natural inteligente YYYYMMDD; gerada a partir da data, compartilhada pelos três papéis.       |
| data           | date / Não         | Data calendário única; derivada de OrderDate, DueDate e ShipDate e preenchida de forma contínua. |
| ano            | smallint / Não     | Ano civil extraído da data.                                                                      |
| trimestre      | smallint / Não     | Trimestre civil 1 a 4; agregação temporal.                                                       |
| mes            | smallint / Não     | Mês numérico 1 a 12.                                                                             |
| dia            | smallint / Não     | Dia do mês 1 a 31.                                                                               |
| dia_semana_iso | smallint / Não     | Dia ISO: 1 segunda-feira a 7 domingo.                                                            |
| inicio_mes     | date / Não         | Primeiro dia do mês; chave de agrupamento mensal.                                                |

Fonte: elaboração própria (2026), com base no DDL e no dicionário do projeto.

Quadro 5 - dw.dim\_produto

| <b>Campo</b>   | <b>Tipo / nulo</b>             | <b>Origem e significado</b>                                                                                 |
|----------------|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| produto_sk     | bigint / Não                   | PK substituta gerada no PostgreSQL; FK da fato.                                                             |
| produto_id     | integer / Não                  | NK única Production.Product.ProductID; lookup e upsert da dimensão.                                         |
| nome           | text / Não                     | Production.Product.Name; descrição atual do produto.                                                        |
| numero_produto | text / Não                     | Production.Product.ProductNumber; código comercial.                                                         |
| cor            | text / Sim                     | Production.Product.Color; NULL preserva ausência.                                                           |
| subcategoria   | text / Não                     | Production.ProductSubcategory.Name; join por ProductSubcategoryID. Ausente vira Sem subcategoria.           |
| categoria      | text / Não                     | Production.ProductCategory.Name via subcategoria; ausente vira Sem categoria.                               |
| ativo_origem   | boolean / Não                  | Metadado SCD1: membro existe atualmente na origem. Não é KPI de atividade comercial.                        |
| atualizado_em  | timestamp with time zone / Não | Instante UTC-equivalente armazenado como timestamptz da última gravação no DW; não é ModifiedDate da fonte. |

Fonte: elaboração própria (2026), com base no DDL e no dicionário do projeto.

Quadro 6 - dw.dim\_territorio

| <b>Campo</b>  | <b>Tipo / nulo</b>             | <b>Origem e significado</b>                                                                                 |
|---------------|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| territorio_sk | bigint / Não                   | PK substituta; membro 0 quando Header.TerritoryID é nulo.                                                   |
| territorio_id | integer / Não                  | NK Sales.SalesTerritory.TerritoryID; 0 é membro técnico.                                                    |
| nome          | text / Não                     | Sales.SalesTerritory.Name; descrição atual do território do pedido.                                         |
| pais_codigo   | text / Não                     | Sales.SalesTerritory.CountryRegionCode; não indica moeda da medida.                                         |
| grupo         | text / Não                     | Sales.SalesTerritory.Group; agrupamento comercial.                                                          |
| ativo_origem  | boolean / Não                  | Metadado SCD1: membro existe atualmente na origem. Não é KPI de atividade comercial.                        |
| atualizado_em | timestamp with time zone / Não | Instante UTC-equivalente armazenado como timestamptz da última gravação no DW; não é ModifiedDate da fonte. |

Fonte: elaboração própria (2026), com base no DDL e no dicionário do projeto.

Quadro 7 - dw.dim\_vendedor

| <b>Campo</b>  | <b>Tipo / nulo</b>             | <b>Origem e significado</b>                                                                                 |
|---------------|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| vendedor_sk   | bigint / Não                   | PK substituta; membro 0 quando Header.SalesPersonID é nulo.                                                 |
| vendedor_id   | integer / Não                  | NK Sales.SalesPerson.BusinessEntityID; 0 é membro técnico.                                                  |
| nome          | text / Não                     | Nome completo de Person.Person ligado a SalesPerson.BusinessEntityID.                                       |
| ativo_origem  | boolean / Não                  | Metadado SCD1: membro existe atualmente na origem. Não é KPI de atividade comercial.                        |
| atualizado_em | timestamp with time zone / Não | Instante UTC-equivalente armazenado como timestamptz da última gravação no DW; não é ModifiedDate da fonte. |

Fonte: elaboração própria (2026), com base no DDL e no dicionário do projeto.

Quadro 8 - dw.fato\_venda

(continua)

| <b>Campo</b>       | <b>Tipo / nulo</b>  | <b>Origem e significado</b>                                                                                |
|--------------------|---------------------|------------------------------------------------------------------------------------------------------------|
| pedido_id          | integer / Não       | SalesOrderHeader/Detail.SalesOrderID; parte da PK e dimensão degenerada do documento.                      |
| item_id            | integer / Não       | SalesOrderDetail.SalesOrderDetailID; parte da PK com pedido_id.                                            |
| data_pedido_sk     | integer / Não       | FK para dim_data, derivada de Header.OrderDate; eixo temporal padrão dos KPIs.                             |
| data_vencimento_sk | integer / Não       | FK para dim_data, derivada de Header.DueDate; marco de vencimento, não prova de entrega.                   |
| data_envio_sk      | integer / Sim       | FK opcional para dim_data, derivada de Header.ShipDate; NULL quando não informada.                         |
| produto_sk         | bigint / Não        | FK para dw.dim_produto. Lookup da NK proveniente do item/cabeçalho; relação N:1.                           |
| cliente_sk         | bigint / Não        | FK para dw.dim_cliente. Lookup da NK proveniente do item/cabeçalho; relação N:1.                           |
| territorio_sk      | bigint / Não        | FK para dw.dim_territorio. Lookup da NK proveniente do item/cabeçalho; relação N:1.                        |
| vendedor_sk        | bigint / Não        | FK para dw.dim_vendedor. Lookup da NK proveniente do item/cabeçalho; relação N:1.                          |
| canal_sk           | smallint / Não      | FK para dw.dim_canal. Lookup da NK proveniente do item/cabeçalho; relação N:1.                             |
| status             | smallint / Não      | Header.Status: 1 processo, 2 aprovado, 3 atraso, 4 rejeitado, 5 expedido, 6 cancelado. Somente 5 nos KPIs. |
| quantidade         | integer / Não       | Detail.OrderQty; inteiro positivo e aditivo.                                                               |
| preco_unitario     | numeric(19,4) / Não | Detail.UnitPrice em UM; não aditivo. Decimal preserva precisão monetária.                                  |
| taxa_desconto      | numeric(9,4) / Não  | Detail.UnitPriceDiscount como fração; intervalo de qualidade 0 a 1; não somável.                           |

Continua na página seguinte.

## Quadro 8 - dw.fato\_venda

(conclusão)

| <b>Campo</b>    | <b>Tipo / nulo</b>             | <b>Origem e significado</b>                                                                                      |
|-----------------|--------------------------------|------------------------------------------------------------------------------------------------------------------|
| receita_liquida | numeric(28,8) / Não            | Detail.LineTotal; preserva valor computado na fonte; soma comercial após desconto, sem frete/impostos.           |
| receita_bruta   | numeric(28,8) / Sim            | Coluna gerada: quantidade × preco_unitario; medida aditiva em UM.                                                |
| valor_desconto  | numeric(28,8) / Sim            | Coluna gerada: receita_bruta – receita_liquida; mantém reconciliação com a fonte.                                |
| versao_origem   | bigint / Não                   | Versão CT do lote da última mudança material carregada no item; não é timestamp nem versão individual do evento. |
| carregado_em    | timestamp with time zone / Não | Instante da inserção ou última atualização material do item no DW.                                               |

Fonte: elaboração própria (2026), com base no DDL e no dicionário do projeto.

## Quadro 9 - etl.controle

| <b>Campo</b>      | <b>Tipo / nulo</b>             | <b>Origem e significado</b>                                                                                 |
|-------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| pipeline          | text / Não                     | PK lógica do processo; valor vendas nesta implementação.                                                    |
| versao            | bigint / Não                   | Última versão CT confirmada junto aos dados, usada no próximo ciclo.                                        |
| identidade_origem | text / Não                     | UUID da preparação e begin_version das tabelas CT; identifica a linhagem esperada.                          |
| atualizado_em     | timestamp with time zone / Não | Instante UTC-equivalente armazenado como timestamptz da última gravação no DW; não é ModifiedDate da fonte. |

Fonte: elaboração própria (2026), com base no DDL e no dicionário do projeto.

Quadro 10 - etl.execucao

| Campo           | Tipo / nulo                    | Origem e significado                                                                                  |
|-----------------|--------------------------------|-------------------------------------------------------------------------------------------------------|
| execucao_id     | uuid / Não                     | UUID gerado em Python; PK para rastrear uma tentativa.                                                |
| inicio          | timestamp with time zone / Não | Instante de registro da tentativa.                                                                    |
| fim             | timestamp with time zone / Sim | Instante de conclusão/falha; nulo enquanto running.                                                   |
| status          | text / Não                     | running, success ou failed; estado operacional da tentativa.                                          |
| modo            | text / Não                     | initial, incremental ou resync; explica se houve leitura inicial/completa explícita.                  |
| versao_anterior | bigint / Sim                   | Checkpoint antes da tentativa; nulo na primeira carga.                                                |
| versao_nova     | bigint / Sim                   | Versão do snapshot confirmada; nula se falhou antes de concluir.                                      |
| estatisticas    | jsonb / Sim                    | JSON com ct_keys, dimension_rows_extracted/written, orders_affected, facts_extracted/written/deleted. |
| erro            | text / Sim                     | Descrição resumida da falha, sem credenciais; nula se não houver erro.                                |

Fonte: elaboração própria (2026), com base no DDL e no dicionário do projeto.

#### Relacionamentos e restrições

A PK da fato é (pedido\_id, item\_id). As NKs descritivas são únicas. São oito FKs; somente data\_envio\_sk admite nulo entre elas. Não há FKs entre dimensões. Quantidade é positiva; preço e receita são não negativos; desconto está entre zero e um; status varia de um a seis. A fórmula de LineTotal admite tolerância de 0,000001 UM.

## APÊNDICE B - SCRIPTS DOS DEZ INDICADORES

SQL PostgreSQL extraído dos arquivos versionados. A primeira view estabelece a população de vendas expedidas; os retornos das dez consultas estão no Apêndice C.

```
CREATE OR REPLACE VIEW dw.v_vendas AS
SELECT f.*, d.data AS data_pedido, d.inicio_mes,
       e.data AS data_envio, v.data AS data_vencimento
FROM dw.fato_venda f
JOIN dw.dim_data d ON d.data_sk=f.data_pedido_sk
JOIN dw.dim_data v ON v.data_sk=f.data_vencimento_sk
LEFT JOIN dw.dim_data e ON e.data_sk=f.data_envio_sk
WHERE f.status=5;

-- KPIs 1 a 8 e 10: todo o período carregado; todos usam Status=5.
-- Para recortes, aplicar o MESMO filtro em numerador e denominador.
CREATE OR REPLACE VIEW dw.kpi_01_receita_bruta AS
SELECT coalesce(sum(receita_bruta),0) AS receita_bruta_um FROM
dw.v_vendas;

CREATE OR REPLACE VIEW dw.kpi_02_descontos AS
SELECT coalesce(sum(valor_desconto),0) AS descontos_um FROM dw.v_vendas;

CREATE OR REPLACE VIEW dw.kpi_03_receita_liquida AS
SELECT coalesce(sum(receita_liquida),0) AS receita_liquida_um FROM
dw.v_vendas;

CREATE OR REPLACE VIEW dw.kpi_04_unidades AS
SELECT coalesce(sum(quantidade),0) AS unidades FROM dw.v_vendas;

CREATE OR REPLACE VIEW dw.kpi_05_pedidos AS
SELECT count(DISTINCT pedido_id) AS pedidos FROM dw.v_vendas;

CREATE OR REPLACE VIEW dw.kpi_06_ticket_medio AS
SELECT sum(receita_liquida)/nullif(count(DISTINCT pedido_id),0) AS
ticket_medio_um
FROM dw.v_vendas;

CREATE OR REPLACE VIEW dw.kpi_07_taxa_desconto AS
SELECT 100.0*sum(valor_desconto)/nullif(sum(receita_bruta),0) AS
desconto_ponderado_pct
FROM dw.v_vendas;

CREATE OR REPLACE VIEW dw.kpi_08_clientes_ativos AS
SELECT count(DISTINCT cliente_sk) AS clientes_com_compra FROM
dw.v_vendas;
```

```

-- Calendário contínuo impede LAG de comparar meses não consecutivos.
CREATE OR REPLACE VIEW dw.kpi_09_crescimento_mensal AS
WITH limites AS (
    SELECT date_trunc('month',min(d.data)) AS primeiro,
           date_trunc('month',max(d.data)) AS ultimo
    FROM dw.fato_venda f JOIN dw.dim_data d ON d.data_sk=f.data_pedido_sk
), meses AS (
    SELECT m::date AS mes FROM limites,
    generate_series(primeiro,ultimo,interval '1 month') m
), totais AS (
    SELECT inicio_mes AS mes,sum(receita_liquida) AS receita FROM
dw.v_vendas GROUP BY inicio_mes
), serie AS (
    SELECT m.mes,coalesce(t.receita,0) AS receita FROM meses m LEFT JOIN
totais t USING(mes)
), anterior AS (
    SELECT mes,receita,lag(receita) OVER (ORDER BY mes) AS receita_anterior
FROM serie
)
SELECT mes,receita,receita_anterior,
    100.0*(receita-receita_anterior)/nullif(receita_anterior,0) AS
crescimento_pct
FROM anterior;

-- Expedição até DueDate: não é entrega ao cliente nem OTIF.
-- DISTINCT remove a repetição de datas do cabeçalho no grão de item.
CREATE OR REPLACE VIEW dw.kpi_10_expedicao_no_prazo AS
WITH pedidos AS (
    SELECT DISTINCT pedido_id,data_envio,data_vencimento FROM dw.v_vendas
)
SELECT count(*) FILTER(WHERE data_envio IS NOT NULL) AS pedidos_com_data,
       count(*) FILTER(WHERE data_envio IS NULL) AS pedidos_sem_data,
       count(*) FILTER(WHERE data_envio<=data_vencimento) AS
pedidos_no_prazo,
       100.0*count(*) FILTER(WHERE data_envio<=data_vencimento)
       /nullif(count(*) FILTER(WHERE data_envio IS NOT NULL),0) AS
expedicao_no_prazo_pct
FROM pedidos;

```

## APÊNDICE C - CONSULTAS E RETORNOS

Resultados de evidence/kpis.json, 12/09/2026, 15:41:54 (UTC-3), versão CT 0. Valores escalares preservam as casas decimais e o ponto do JSON. A série mensal apresenta receitas com seis casas e crescimento com quatro casas decimais; NULL indica ausência de base válida.

### C.1 Receita bruta

```
SELECT * FROM dw.kpi_01_receita_bruta;
```

| Campo retornado  | Valor              |
|------------------|--------------------|
| receita_bruta_um | 110373889.31340000 |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.

### C.2 Descontos comerciais

```
SELECT * FROM dw.kpi_02_descontos;
```

| Campo retornado | Valor           |
|-----------------|-----------------|
| descontos_um    | 527507.91351200 |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.

### C.3 Receita líquida comercial

```
SELECT * FROM dw.kpi_03_receita_liquida;
```

| Campo retornado    | Valor              |
|--------------------|--------------------|
| receita_liquida_um | 109846381.39988800 |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.

### C.4 Unidades vendidas

```
SELECT * FROM dw.kpi_04_unidades;
```

| Campo retornado | Valor  |
|-----------------|--------|
| unidades        | 274914 |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.

#### C.5 Pedidos expedidos

```
SELECT * FROM dw.kpi_05_pedidos;
```

| Campo retornado | Valor |
|-----------------|-------|
| pedidos         | 31465 |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.

#### C.6 Ticket médio

```
SELECT * FROM dw.kpi_06_ticket_medio;
```

| Campo retornado | Valor                 |
|-----------------|-----------------------|
| ticket_medio_um | 3491.0656729664071190 |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.

#### C.7 Taxa ponderada de desconto

```
SELECT * FROM dw.kpi_07_taxa_desconto;
```

| Campo retornado        | Valor                  |
|------------------------|------------------------|
| desconto_ponderado_pct | 0.47792817376777681849 |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.

#### C.8 Clientes com compra

```
SELECT * FROM dw.kpi_08_clientes_ativos;
```

| Campo retornado     | Valor |
|---------------------|-------|
| clientes_com_compra | 19119 |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.

### C.9 Crescimento mensal

SELECT \* FROM dw.kpi\_09\_crescimento\_mensal ORDER BY mes;

Tabela C.9 - Crescimento mensal

(continua)

| Mês        | Receita (UM)     | Receita anterior (UM) | Crescimento (%) |
|------------|------------------|-----------------------|-----------------|
| 2011-05-01 | 503.805,916900   | NULL                  | NULL            |
| 2011-06-01 | 458.910,824800   | 503.805,916900        | -8,9112         |
| 2011-07-01 | 2.044.600,003338 | 458.910,824800        | 345,5332        |
| 2011-08-01 | 2.495.816,733446 | 2.044.600,003338      | 22,0687         |
| 2011-09-01 | 502.073,845800   | 2.495.816,733446      | -79,8834        |
| 2011-10-01 | 4.588.761,816130 | 502.073,845800        | 813,9615        |
| 2011-11-01 | 737.839,821400   | 4.588.761,816130      | -83,9207        |
| 2011-12-01 | 1.309.863,251140 | 737.839,821400        | 77,5268         |
| 2012-01-01 | 3.970.627,278958 | 1.309.863,251140      | 203,1330        |
| 2012-02-01 | 1.475.426,909980 | 3.970.627,278958      | -62,8415        |
| 2012-03-01 | 2.975.748,238428 | 1.475.426,909980      | 101,6873        |
| 2012-04-01 | 1.634.600,798332 | 2.975.748,238428      | -45,0693        |
| 2012-05-01 | 3.074.602,813538 | 1.634.600,798332      | 88,0950         |
| 2012-06-01 | 4.099.354,356751 | 3.074.602,813538      | 33,3296         |
| 2012-07-01 | 3.417.953,869571 | 4.099.354,356751      | -16,6221        |
| 2012-08-01 | 2.175.637,217725 | 3.417.953,869571      | -36,3468        |
| 2012-09-01 | 3.454.151,940485 | 2.175.637,217725      | 58,7651         |
| 2012-10-01 | 2.544.091,105517 | 3.454.151,940485      | -26,3469        |
| 2012-11-01 | 1.872.701,976206 | 2.544.091,105517      | -26,3901        |
| 2012-12-01 | 2.829.404,818943 | 1.872.701,976206      | 51,0868         |
| 2013-01-01 | 2.087.872,462504 | 2.829.404,818943      | -26,2081        |
| 2013-02-01 | 2.316.922,151480 | 2.087.872,462504      | 10,9705         |
| 2013-03-01 | 3.412.068,967535 | 2.316.922,151480      | 47,2673         |

Continua na página seguinte.

Tabela C.9 - Crescimento mensal

(conclusão)

| Mês        | Receita (UM)     | Receita anterior (UM) | Crescimento (%) |
|------------|------------------|-----------------------|-----------------|
| 2013-04-01 | 2.532.265,912399 | 3.412.068,967535      | -25,7850        |
| 2013-05-01 | 3.245.623,754479 | 2.532.265,912399      | 28,1707         |
| 2013-06-01 | 5.081.069,131596 | 3.245.623,754479      | 56,5514         |
| 2013-07-01 | 4.896.353,737794 | 5.081.069,131596      | -3,6354         |
| 2013-08-01 | 3.333.964,067555 | 4.896.353,737794      | -31,9092        |
| 2013-09-01 | 4.532.908,705303 | 3.333.964,067555      | 35,9615         |
| 2013-10-01 | 4.795.813,289802 | 4.532.908,705303      | 5,7999          |
| 2013-11-01 | 3.312.130,245587 | 4.795.813,289802      | -30,9370        |
| 2013-12-01 | 4.075.486,625601 | 3.312.130,245587      | 23,0473         |
| 2014-01-01 | 4.289.817,950953 | 4.075.486,625601      | 5,2590          |
| 2014-02-01 | 1.337.725,035600 | 4.289.817,950953      | -68,8163        |
| 2014-03-01 | 7.217.531,091974 | 1.337.725,035600      | 439,5377        |
| 2014-04-01 | 1.797.173,923000 | 7.217.531,091974      | -75,0999        |
| 2014-05-01 | 5.366.674,969338 | 1.797.173,923000      | 198,6175        |
| 2014-06-01 | 49.005,840000    | 5.366.674,969338      | -99,0868        |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.

## C.10 Expedição até vencimento

SELECT \* FROM dw.kpi\_10\_expedicao\_no\_prazo;

| Campo retornado        | Valor                |
|------------------------|----------------------|
| pedidos_com_data       | 31465                |
| pedidos_sem_data       | 0                    |
| pedidos_no_prazo       | 31465                |
| expedicao_no_prazo_pct | 100.0000000000000000 |

Fonte: evidence/kpis.json; 12/09/2026, 15:41:54 (UTC-3), versão CT 0.